

LISTA DOBLEMENTE ENLAZADA

PROGRAMACION III

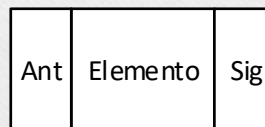
INF - 131

Lic. Marcelo Aruquipa

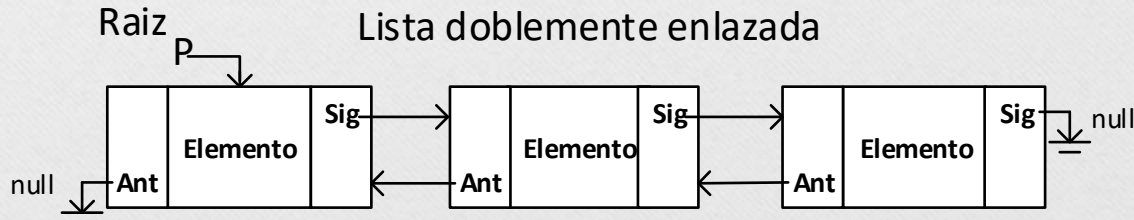
LISTA DOBLEMENTE ENLAZADA

- Es una colección de nodos
- El nodo tiene tres partes
 - Un enlace que apunta al anterior nodo.
 - El elemento (información)
 - Un enlace que apunta al siguiente nodo

Nodo



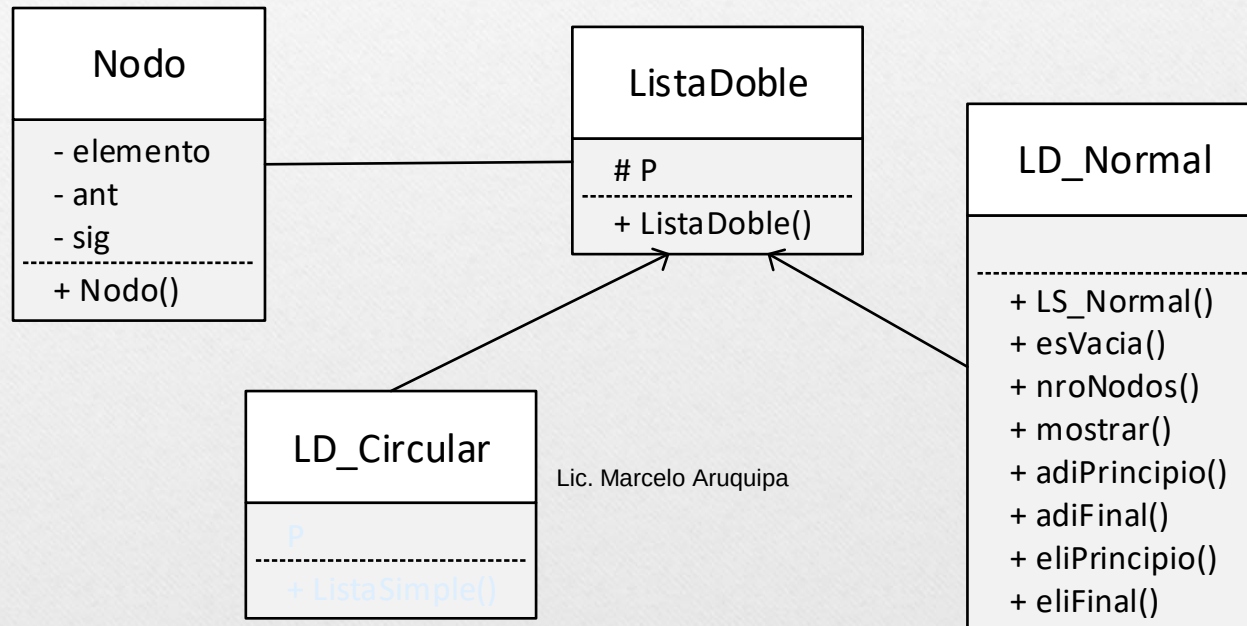
Lic. Marcelo Aruquipa



LISTA DOBLE

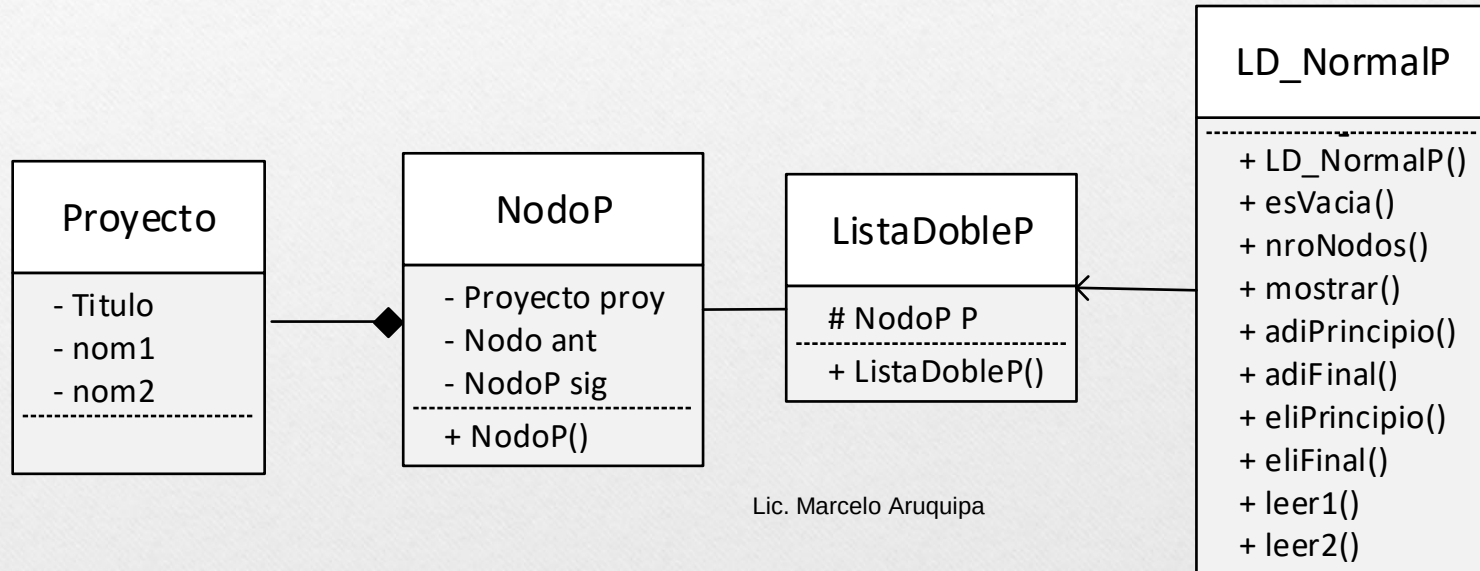
- Existen dos tipos de listas dobles

Diagrama de Clases



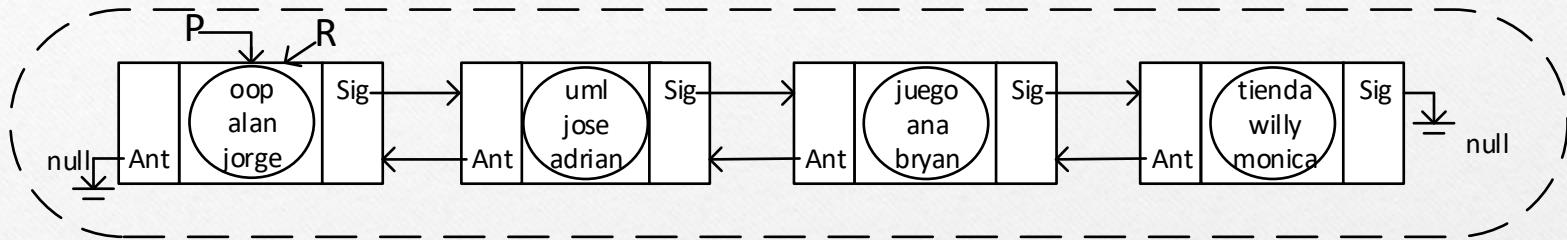
LISTA DOBLE NORMAL

Ejemplo. Implementar una lista doble de objetos Proyecto.



LISTA DOBLE NORMAL

Objeto de la clase LD_NormalP



Recorrer la lista

Se puede recorrer a la izquierda(ant) ó derecha(izq):

$$R \leftarrow R.\text{ant} \quad , \quad R \leftarrow R.\text{sig}$$

*“R tiene la dirección del anterior **Nodo**”* , *“R tiene la dirección del siguiente **Nodo**”*

*“R apunta al anterior **Nodo**”* , *“R apunta al siguiente **Nodo**”*

Lic. Marcelo Aruquipa

LISTA DOBLE NORMAL

Clase NodoP

```
class NodoP
{
  private:
    Proyecto proy;
    NodoP ant;
    NodoP sig;
  public:
    NodoP()
    {
      sig <- null;
      ant <- null
    }
}
```

Clase ListaDobleP

```
class ListaDobleP
{
  protected:
    NodoP P;
  public:
    ListaDobleP()
    {
      P <- null;
    }
}
```

Lic. Marcelo Aruquipa

Se asume la implementación de la **clase Proyecto** (atributos y métodos)

LISTA DOBLE NORMAL

Clase LD_NormalP

```
class LD_NormalP : ListaDobleP
{
    public:
        LD_NormalP()
        {
            ListaDobleP();
        }

        bool esVacia()
        {
            if(P == null)
                return true;
            return false;
        }

        int nroNodos()
        {
            int c = 0;
            NodoP R = P;
            while(R != null)
            {
                c++;
                R = R.getSig();
            }
            return c;
        }
}
```

```
adiPrimero(Proyecto z)
{
    nuevo = new NodoP();
    nuevo.setProy(z);
    if(esVacia())
        P = nuevo;
    else{
        nuevo.setSig(P);
        P.setAnt(nuevo);
        P = nuevo;
    }
}
```

Lic. Marcelo Aruquipa

```
mostrar()
{
    NodoP R = P;
    while(R != null)
    {
        R.getProy().mostrar();
        R = R.getSig();
    }
}

adiFinal(Proyecto z)
{
    nuevo = new NodoP();
    nuevo.setProy(z);
    if(esVacia())
        P = nuevo;
    else{
        NodoP R = P;
        while(R.getSig() != null)
        {
            R = R.getSig();
        }
        R.setSig(nuevo);
        nuevo.setAnt(R);
    }
}
```

LISTA DOBLE NORMAL

Clase LD_NormalP

```
NodoP eliPrimero()
{
    x <- new NodoP();
    if(!esVacia())
    {
        if(nroNodos() = 1){
            x <- P;
            P <- null;
        }else{
            x <- P;
            P <- P.getSig();
            P.setAnt(null);
            x.setSig(null);
        }
    }
    return x;
}
```

```
NodoP eliFinal()
{
    x <- new NodoP();
    if(!esVacia())
    {
        if(nroNodos() = 1)
        {
            x <- P;
            P <- null;
        }
        else{
            NodoP R <- P;
            while( R.getSig() != null)
            {
                R <- R.getSig();
            }
            x <- R;
            NodoP aux <- R.getAnt();
            aux.setSig(null);
            x.setAnt(null);
        }
    }
    return x;
}
```


LISTA DOBLE NORMAL

Clase LD_NormalP

```
leer1(int n)
{
    for (i <- 1 to n)
    {
        z <- new Proyecto();
        z.leer();
        adiPrimero(z);
    }
}

leer2(int n)
{
    for (i <- 1 to n)
    {
        z <- new Proyecto();
        z.leer();
        adiFinal(z);
    }
}

} //fin clase LD_NormalP
```

```
main()
{
    A <- new LD_NormalP();
    A.leer1(5);
    A.mostrar();

    Proyecto z <- A.eliPrimero();
    z.mostrar();

    A.adiUltimo(z);
    A.mostrar();
}
```

Lic. Marcelo Aruquipa

Preguntas?

Ver el archivo adjunto